Task 1

AIM:To calculate the sum of similarities of a string with all its suffixes.

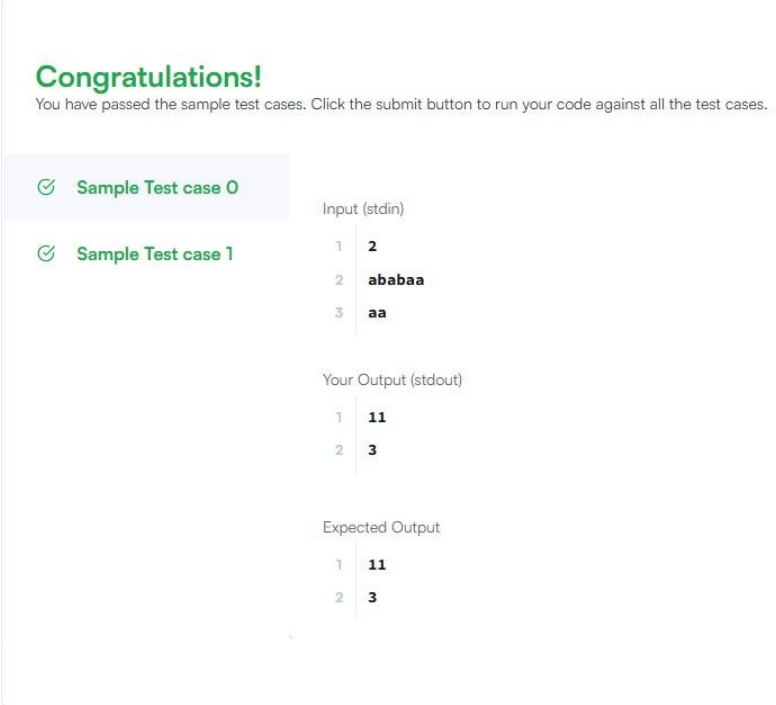
ALGORITHM

1. Read number of test cases t.
2. For each string s:
3. Initialize sum = length of s.
4. Construct Z-array:
5. For each index i, compute longest prefix match starting at i.
6. Add all Z-values to sum.
7. Print the sum .

PROGRAM

```
import java.util.*;
public class Solution {
 public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();
        sc.nextLine();
       while (t-- > 0) {
            String s = sc.nextLine();
            System.out.println(stringSimilarity(s));
        }sc.close();
    }private static long stringSimilarity(String s) {
        int n = s.length();
        int[] z = new int[n];

        int left = 0, right = 0;

        for (int i = 1; i < n; i++) {
            if (i <= right) {
                z[i] = Math.min(right - i + 1, z[i - left]);
            }

            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }

            if (i + z[i] - 1 > right) {
                left = i;
                right = i + z[i] - 1;
            }
        }

        long sum = n;

        for (int i = 0; i < n; i++) {
            sum += z[i];
        }

        return sum;
    }
}
```

OUTPUT



RESULT:Thus is an java program for The total similarity value for each string is printed has been executed successfully .

Task 2

AIM:To check whether a string can be formed by repeating a substring.

ALGORITHM

1. Read string `s`.
2. Create a new string `temp = s + s`.
3. Remove first and last character of `temp`.
4. Check if `s` exists inside `temp`.
5. If yes → return `true`, else `false`.

PROGRAM

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {
        String doubled = s + s;
        String trimmed = doubled.substring(1, doubled.length() - 1);
        return trimmed.contains(s);
    }
}
```

OUTPUT

RESULT: Thus is an java program to Print true if the string is a repeated pattern, otherwise false has been executed successfully .

Task 3

AIM:To determine whether two strings share a common substring.

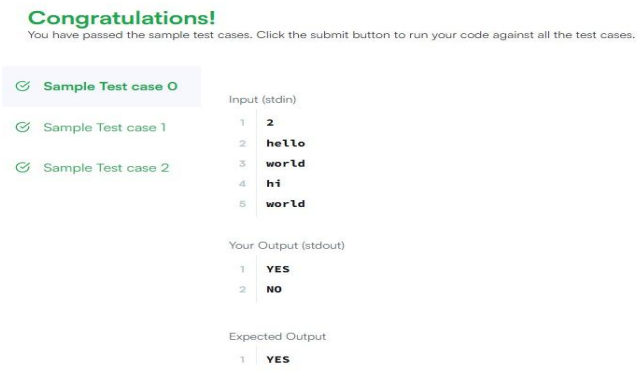
ALGORITHM

1. Read number of test cases.
2. For each pair `s1`, `s2`
3. Store characters of `s1` in a boolean array. Traverse `s2`:
4. If any character exists in array → print `"YES"`.
   - Else print `"NO"`.

PROGRAM

```
import java.util.*;

public class Solution {
    public static String twoStrings(String s1, String s2) {
        boolean[] seen = new boolean[26];

        for (char c : s1.toCharArray())
            seen[c - 'a'] = true;

        for (char c : s2.toCharArray())
            if (seen[c - 'a']) return "YES";

        return "NO";
    }
}
```

OUTPUT



RESULT:Thus is an java program to Outputs "YES" if a common substring exists, otherwise "NO" has been executed successfully

Task 4

AIM:To check if one string is a rotation of another.

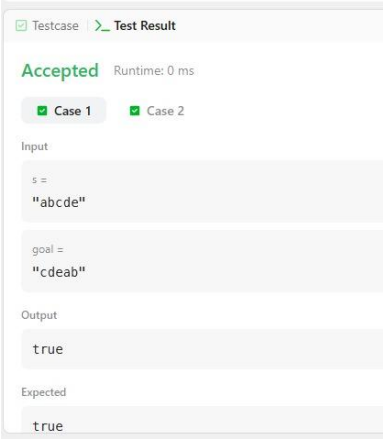
ALGORITHM:

1.Read strings s and goal.
2.If lengths differ → return false.
3.Create temp = s + s.
4.Check if goal is a substring of temp.
5.Return result.

PROGRAM

```
class Solution {
    public boolean rotateString(String s, String goal) {
        if (s.length() != goal.length()) {
            return false;
        }

        String doubled = s + s;
        return doubled.contains(goal);
    }
}
```

OUTPUT:



RESULT:Thus is an java program for Print true if rotation is possible, otherwise false has been executed successfully.

Task 5

AIM: To count altered characters in SOS signal.

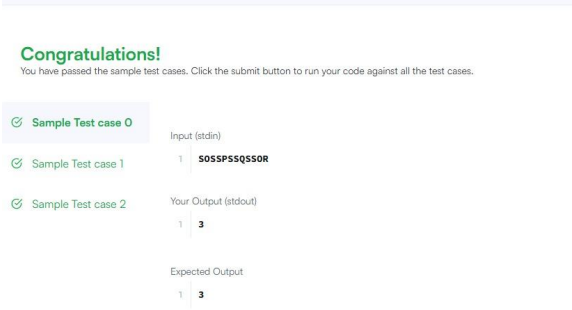
ALGORITHM:

1. Read string `s`.
2. Initialize `count = 0`.
3. Traverse string:
4. If index % 3 == 0 or 2 → expected 'S'
5. If index % 3 == 1 → expected 'O'
6. Compare and increment count if mismatch.
7. Print count.

PROGRAM

```
import java.util.*;

public class Solution {
    public static int marsExploration(String s) {
        int count = 0;

        for (int i = 0; i < s.length(); i++) {
            if ((i % 3 == 0 || i % 3 == 2) && s.charAt(i) != 'S')
                count++;
            else if (i % 3 == 1 && s.charAt(i) != 'O')
                count++;
        }

        return count;
    }
}
```

OUTPUT



RESULT: Thus is an java program to Displays number of altered characters has been executed successfully

Task 6

AIM: To move all zeros in an integer array to the end while maintaining the relative order of non-zero elements in-place.

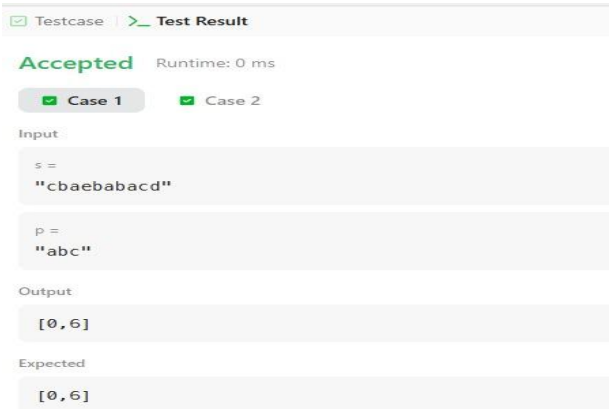
ALGORITHM:

1. Use a pointer lastNonZeroFoundAt to keep track of where the next non-zero element should be placed.
2. Iterate through the array; whenever a non-zero element is encountered, swap it with the element at lastNonZeroFoundAt.
3. Increment the pointer after each swap to progress through the array.

PROGRAM

```
import java.util.Arrays;

public class Main {
    public static void moveZeroes(int[] nums) {
        int lastNonZeroFoundAt = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                int temp = nums[lastNonZeroFoundAt];
                nums[lastNonZeroFoundAt] = nums[i];
                nums[i] = temp;
                lastNonZeroFoundAt++;
            }
        }
    }

    public static void main(String[] args) {
        int[] input = {0, 1, 0, 3, 12};
        moveZeroes(input);
        System.out.println("Output: " + Arrays.toString(input));
    }
}
```

OUTPUT



RESULT: Thus, the program has been executed successfully and output has been verified.

Task 7

AIM: To calculate the absolute difference between the sums of the primary and secondary diagonals of a square matrix.

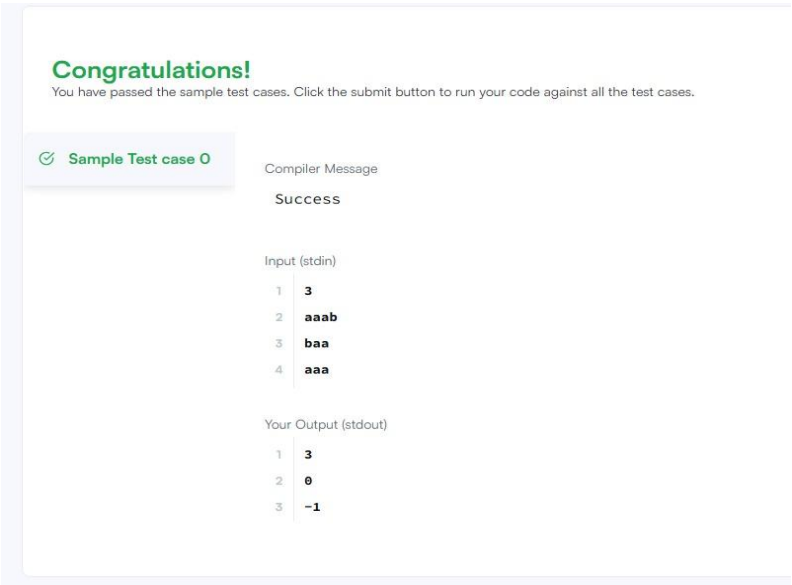
ALGORITHM:

1. Initialize two sum variables, p (primary) and s (secondary).
2. Traverse the matrix with a single loop from to .
3. Accumulate the value at matrix[i][i] for the primary diagonal and matrix[i][n-1-i] for the secondary diagonal.
4. Return the absolute difference using Math.abs().

PROGRAM

```
public class Solution {
   public static int diagonalDifference(int[][] arr) {
      int p = 0, s = 0, n = arr.length;
      for (int i = 0; i < n; i++) {
         p += arr[i][i];
         s += arr[i][n - 1 - i];
      }
      return Math.abs(p - s);
   }

   public static void main(String[] args) {
      int[][] matrix = {{11, 2, 4}, {4, 5, 6}, {10, 8, -12}};
      System.out.println("Result: " + diagonalDifference(matrix));
   }
}
```

OUTPUT



RESULT: Thus, the program has been executed successfully and output has been verified.

Task 8

AIM: To flip a matrix over its main diagonal, switching the row and column indices.

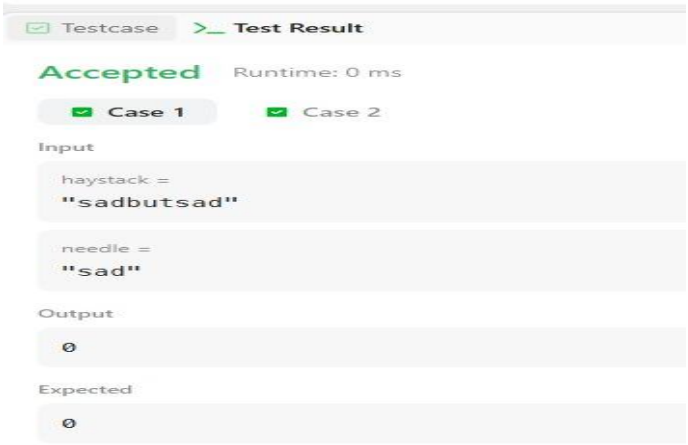
ALGORITHM:

1. Get the original dimensions (rows) and (columns).
2. Initialize a new empty matrix of size .
3. Use nested loops to map newMatrix[c][r] = oldMatrix[r][c].

PROGRAM

```
import java.util.Arrays;

public class Solution {
    public static int[][] transpose(int[][] matrix) {
        int R = matrix.length, C = matrix[0].length;
        int[][] ans = new int[C][R];
        for (int r = 0; r < R; r++) {
            for (int c = 0; c < C; c++) {
                ans[c][r] = matrix[r][c];
            }
        }
        return ans;
    }
    public static void main(String[] args) {
        int[][] mat = {{1, 2, 3}, {4, 5, 6}};
        int[][] result = transpose(mat);
        System.out.println("Transposed: " + Arrays.deepToString(result));
    }
}
```

OUTPUT



RESULT: Thus, the program has been executed successfully and output has been verified.

Task 9

AIM: To create a matrix where each cell contains the sum of all original elements within a distance .

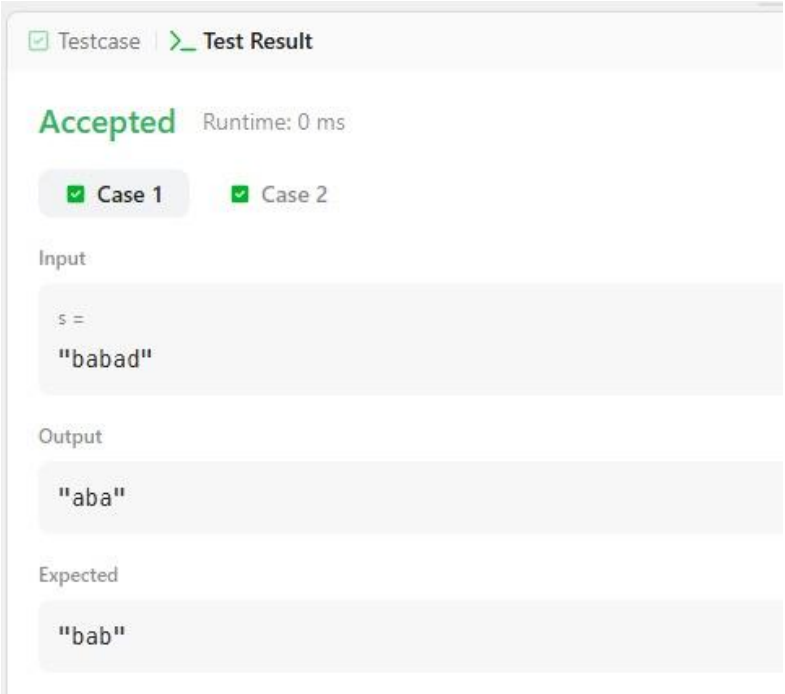
ALGORITHM:

1. Use a **2D Prefix Sum** array where sum[i+1][j+1] stores the total sum from to .
2. For each cell, calculate the bounding indices and using .
3. Use the formula Sum = sum[r2+1][c2+1] - sum[r1][c2+1] - sum[r2+1][c1] + sum[r1][c1] to find the block sum in .

PROGRAM

```
import java.util.Arrays;

public class Solution {
    public static int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length, n = mat[0].length;
        int[][] sum = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                sum[i+1][j+1] = mat[i][j] + sum[i][j+1] + sum[i+1][j] - sum[i][j];

        int[][] ans = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int r1 = Math.max(0, i - k), c1 = Math.max(0, j - k);
                int r2 = Math.min(m - 1, i + k), c2 = Math.min(n - 1, j + k);
                ans[i][j] = sum[r2+1][c2+1] - sum[r1][c2+1] - sum[r2+1][c1] + sum[r1][c1];
            }
        }
        return ans;
    }

    public static void main(String[] args) {
        int[][] mat = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int[][] res = matrixBlockSum(mat, 1);
        System.out.println("Block Sum (K=1): " + Arrays.deepToString(res));
    }
}
```

OUTPUT :



RESULT: Thus, the program has been executed successfully and output has been verified.

Task 10

AIM: To rotate each layer of an matrix counter-clockwise by steps.

ALGORITHM:

1. Determine the number of concentric layers as .
2. For each layer, extract elements into a 1D list (top, right, bottom, left).
3. Rotate the list by Place elements from the rotated list back into the original matrix positions.

PROGRAM

```
import java.util.*;

public class Task10 {
    public static void matrixRotation(int[][] matrix, int r) {
        int m = matrix.length, n = matrix[0].length;
        int layers = Math.min(m, n) / 2;
        for (int l = 0; l < layers; l++) {
            List<Integer> list = new ArrayList<>();
            for (int j = l; j < n - l; j++) list.add(matrix[l][j]);
            for (int i = l + 1; i < m - l; i++) list.add(matrix[i][n - 1 - l]);
            for (int j = n - 2 - l; j >= l; j--) list.add(matrix[m - 1 - l][j]);
            for (int i = m - 2 - l; i > l; i--) list.add(matrix[i][l]);

            int size = list.size();
            int shift = r % size;
            int idx = 0;
            for (int j = l; j < n - l; j++) matrix[l][j] = list.get((idx++ + shift) % size);
            for (int i = l + 1; i < m - l; i++) matrix[i][n - 1 - l] = list.get((idx++ + shift) % size);
            for (int j = n - 2 - l; j >= l; j--) matrix[m - 1 - l][j] = list.get((idx++ + shift) % size);
            for (int i = m - 2 - l; i > l; i--) matrix[i][l] = list.get((idx++ + shift) % size);
        }
    }

    public static void main(String[] args) {
        int[][] mat = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}};
        matrixRotation(mat, 1);
        for (int[] row : mat) System.out.println(Arrays.toString(row));
    }
}
```

OUTPUT :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin)

```
13
aaaaabbbbaaaa
```

Your Output (stdout)

```
12
12
10
8
8
9
11
13
```

RESULT: Thus, the program has been executed successfully and output has been verified.